

SAP ABAP — Unit 6, Exercise 15

Use a Complex Internal Table

Complete practice notes • Tasks, solution, screenshots, concepts, why/how explanations

1. Exercise Objective

The goal of Exercise 15 is to learn how to store and access multiple structured records inside an ABAP internal table. Instead of keeping only one airport record, the class loads all available airports from the CDS view `/DMO/I_Airport` into a static internal table and then looks up the departure and destination airport names.

Main idea: One structure represents one airport; an internal table represents many airports.

2. What You Build

The final design is:

```
st_airport
├── AirportID
└── Name
```

```
tt_airports
└── STANDARD TABLE OF st_airport
```

```
CLASS-DATA airports TYPE tt_airports
```

```
/DMO/I_Airport
↓
SELECT all airports
↓
airports internal table
↓
table expression using AirportID
↓
departure / destination structures
↓
get_output( )
↓
ABAP Console
```

3. Template and Solution

- Template: `/LRN/CL_S4D400_STS_STRUCTURE`
- Solution: `/LRN/CL_S4D400_ITS_ITAB`
- Suggested personal class: `ZCL_93_ITAB` (replace 93 with your group number if required).

4. Task 1 — Copy the Template

You can either duplicate /LRN/CL_S4D400_STS_STRUCTURE or continue from your completed Exercise 14 class. The purpose is to preserve the previous connection logic and extend it with an internal table of airports.

Steps:

- Open /LRN/CL_S4D400_STS_STRUCTURE in Eclipse.
- Link the Project Explorer with the editor if necessary.
- Right-click the class in Project Explorer.
- Choose Duplicate.
- Enter your package and a class name such as ZCL_##_ITAB.
- Adjust the description.
- Complete the duplicate operation and transport request.

5. Task 2 — Declare the Internal Table

The first design decision is to describe what one airport looks like. That is the structure `st_airport`. Then we create `tt_airports`, which says that the program can hold many `st_airport` records.

5.1 Define `st_airport`

```
TYPES:
  BEGIN OF st_airport,
    AirportID TYPE /dmo/airport_id,
    Name      TYPE /dmo/airport_name,
  END OF st_airport.
```

Why: The structure is the line type of the internal table. Each row contains an airport ID and its name.

5.2 Define `tt_airports`

```
TYPES tt_airports TYPE STANDARD TABLE OF st_airport
      WITH NON-UNIQUE DEFAULT KEY.
```

Meaning:

- **STANDARD TABLE:** records are stored as a standard internal table and are normally accessed sequentially or by table expression.
- **OF `st_airport`:** every row has the structure `st_airport`.
- **WITH NON-UNIQUE DEFAULT KEY:** the table uses the default key and duplicate key values are allowed.

5.3 Declare the Static Attribute

```
CLASS-DATA airports TYPE tt_airports.
```

Why CLASS-DATA: The airport list is common reference data. Every `lcl_connection` object can use the same airport table instead of creating a separate copy.

6. Task 3 — Fill the Static Attribute in a Class Constructor

A class constructor is executed when the class is initialized. Here it is used to load the airport reference data once into the static attribute `airports`.

6.1 Generate the Class Constructor

In the local class:

- Place the cursor on the class name lcl_connection.
- Press Ctrl + 1.
- Choose Generate class constructor.
- Eclipse creates METHOD class_constructor.

6.2 SELECT All Airports

```
METHOD class_constructor.
```

```
    SELECT FROM /DMO/I_Airport  
    FIELDS AirportID, Name  
    INTO TABLE @airports.
```

```
ENDMETHOD.
```

This reads all rows from /DMO/I_Airport. AirportID and Name are placed into the corresponding columns of the airports internal table.

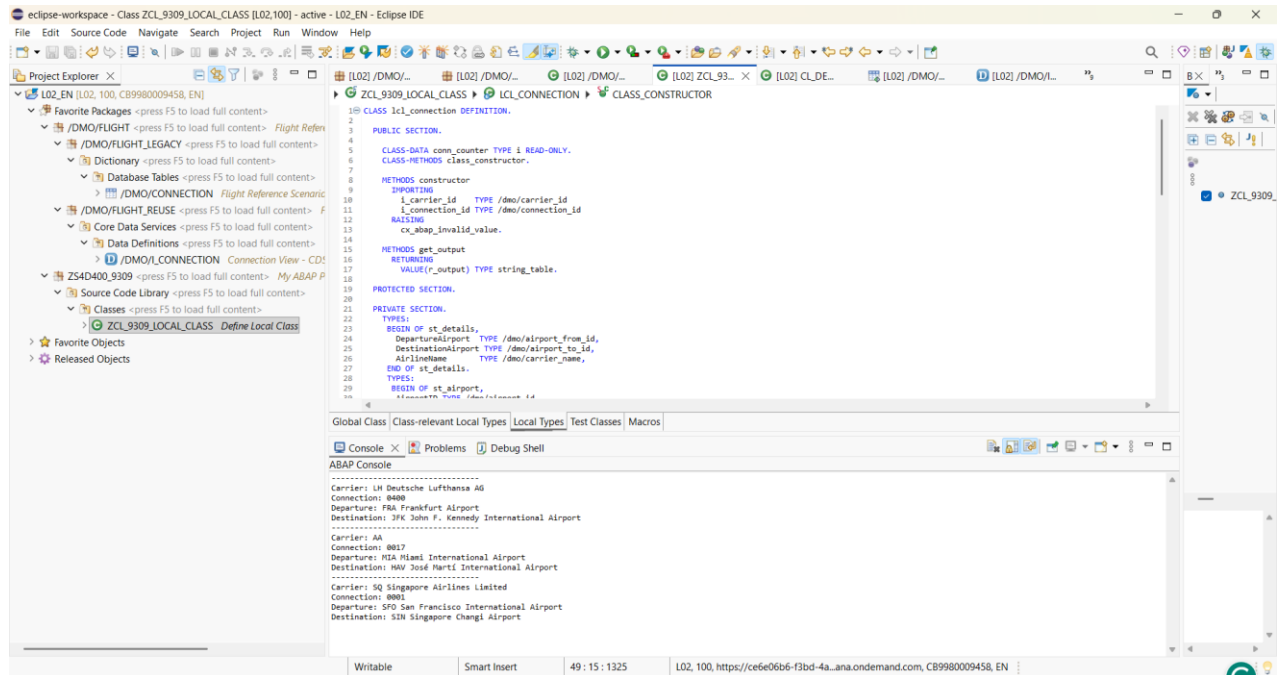
6.3 Understand Each Part

```
    SELECT FROM /DMO/I_Airport  
    FIELDS AirportID, Name  
    INTO TABLE @airports.
```

- SELECT FROM /DMO/I_Airport — read data from the CDS view entity.
- FIELDS AirportID, Name — select only the two fields needed.
- INTO TABLE @airports — put all selected rows into the internal table.
- @airports — @ identifies the ABAP data object in the modern Open SQL syntax.

Important: Unlike SELECT SINGLE, this SELECT reads multiple records. Therefore INTO TABLE is required.

7. Screenshot — Your Task 3 / Class Constructor Result



Screenshot from the completed exercise showing the local class and successful airport output.

8. Task 4 — Access the Internal Table

Now the table contains airport records. The next step is to find the name of the departure and destination airports using the airport IDs already stored in details.

8.1 Read the Departure Airport

```
DATA(departure) = airports[
    airportid = details-DepartureAirport
].
```

This is a table expression. It searches the internal table for the row whose AirportID matches the departure airport ID.

8.2 Read the Destination Airport

```
DATA(destination) = airports[
    airportid = details-DestinationAirport
].
```

The same lookup is performed for the destination airport.

8.3 Why Use Inline DATA?

```
DATA(departure) = ...
DATA(destination) = ...
```

ABAP infers the structure type from the table expression result. You do not need to declare the complete type manually.

8.4 Use the Structure Components

```
APPEND |Departure: { details-DepartureAirport } { departure-Name }| TO r_output.
APPEND |Destination: { details-DestinationAirport } { destination-Name }| TO r_output.
```

departure-Name and destination-Name contain the readable airport names obtained from the internal table.

9. Complete get_output Method

```
METHOD get_output.
```

```
DATA(departure) = airports[  
    airportid = details-DepartureAirport  
].
```

```
DATA(destination) = airports[  
    airportid = details-DestinationAirport  
].
```

```
APPEND |-----| TO r_output.  
APPEND |Carrier: { carrier_id } { details-AirlineName }| TO r_output.  
APPEND |Connection: { connection_id }| TO r_output.  
APPEND |Departure: { details-DepartureAirport } { departure-Name }| TO r_output.  
APPEND |Destination: { details-DestinationAirport } { destination-Name }| TO r_output.
```

```
ENDMETHOD.
```

Your screenshot shows this approach working successfully. The console displays both the airport IDs and the corresponding full airport names.

10. Full Important Local-Class Definition

```
CLASS lcl_connection DEFINITION.
```

```
    PUBLIC SECTION.
```

```
        CLASS-DATA conn_counter TYPE i READ-ONLY.
```

```
        CLASS-METHODS class_constructor.
```

```
        METHODS constructor
```

```
            IMPORTING
```

```
                i_carrier_id    TYPE /dmo/carrier_id
```

```
                i_connection_id TYPE /dmo/connection_id
```

```
            RAISING
```

```
                cx_abap_invalid_value.
```

```
        METHODS get_output
```

```
            RETURNING
```

```
                VALUE(r_output) TYPE string_table.
```

```
    PROTECTED SECTION.
```

```
    PRIVATE SECTION.
```

```
    TYPES:
```

```
        BEGIN OF st_details,
```

```
            DepartureAirport TYPE /dmo/airport_from_id,
```

```
            DestinationAirport TYPE /dmo/airport_to_id,
```

```
            AirlineName       TYPE /dmo/carrier_name,
```

```
        END OF st_details.
```

```

TYPES:
  BEGIN OF st_airport,
    AirportID TYPE /dmo/airport_id,
    Name      TYPE /dmo/airport_name,
  END OF st_airport.

TYPES tt_airports TYPE STANDARD TABLE OF st_airport
      WITH NON-UNIQUE DEFAULT KEY.

DATA carrier_id    TYPE /dmo/carrier_id.
DATA connection_id TYPE /dmo/connection_id.
DATA details       TYPE st_details.

CLASS-DATA airports TYPE tt_airports.

ENDCLASS.

```

Note: The exact declaration can vary slightly depending on what your Exercise 14 template already contains. The important new parts for Exercise 15 are st_airport, tt_airports, CLASS-DATA airports, and class_constructor.

11. Complete Class Constructor

```

CLASS lcl_connection IMPLEMENTATION.

  METHOD class_constructor.

    SELECT FROM /DMO/I_Airport
      FIELDS AirportID, Name
      INTO TABLE @airports.

  ENDMETHOD.

  METHOD constructor.

    IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    SELECT SINGLE
      FROM /DMO/I_Connection
      FIELDS DepartureAirport,
              DestinationAirport,
              \_Airline-Name
      WHERE AirlineID = @i_carrier_id
      AND ConnectionID = @i_connection_id
      INTO @details.

    IF sy-subrc <> 0.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    me->carrier_id = i_carrier_id.
    me->connection_id = i_connection_id.

    conn_counter = conn_counter + 1.
  
```

ENDMETHOD.

ENDCLASS.

The constructor above is included to show how Exercise 15 builds on Exercise 14. The class constructor loads the airport master/reference data; the instance constructor loads the connection-specific details.

12. How the Whole Program Works

1. ABAP initializes `lcl_connection`
↓
2. `class_constructor` executes
↓
3. `/DMO/I_Airport` is read
↓
4. All airport rows are stored in CLASS-DATA `airports`
↓
5. Main program creates a connection object
↓
6. constructor reads the connection from `/DMO/I_Connection`
↓
7. `details` contains departure/destination IDs
↓
8. `get_output` searches airports using those IDs
↓
9. Airport names are retrieved
↓
10. Output is written to the ABAP Console

13. Why Not Just Store the Airport Names in details?

You could retrieve the airport names directly in the connection SELECT, but this exercise is teaching a different concept: a reusable internal table. The airport list is reference data that can be searched repeatedly. Keeping it in a static internal table demonstrates how an ABAP program can buffer and reuse a collection of structured records.

14. Table Expression Explained

```
airports[  
    airportid = details-DepartureAirport  
]
```

Read it from left to right:

- `airports` — the internal table.
- `[ ... ]` — table expression; find one row.
- `airportid` — component of the table row.
- `details-DepartureAirport` — value used for the lookup.
- Result — one `st_airport` structure.

Potential runtime issue: If no matching airport exists, a table expression can raise a table-expression exception. In production code, you should consider appropriate handling when the data may be missing.

15. Optional Direct Table Expression

The exercise also gives an alternative: you can skip the temporary departure and destination structures and access the table directly inside the string template.

```
APPEND |Departure: { details-DepartureAirport }  
{ airports[ airportid = details-DepartureAirport ]-Name }| TO r_output.
```

```
APPEND |Destination: { details-DestinationAirport }  
{ airports[ airportid = details-DestinationAirport ]-Name }| TO r_output.
```

Recommendation: For learning and debugging, the departure/destination variables are clearer. Direct table expressions are shorter but harder to inspect while debugging.

16. Activation and Execution Steps

- Save the source.
- Press Ctrl + F3 to activate.
- Check the Problems view. There should be no activation errors.
- Press F9 to execute the ABAP class.
- Open the ABAP Console.
- Verify that each connection shows carrier, connection ID, departure airport ID/name, and destination airport ID/name.

17. Expected Console Output

```
-----  
Carrier: LH Deutsche Lufthansa AG  
Connection: 0400  
Departure: FRA Frankfurt Airport  
Destination: JFK John F. Kennedy International Airport
```

```
-----  
Carrier: AA  
Connection: 0017  
Departure: MIA Miami International Airport  
Destination: HAV José Martí International Airport
```

```
-----  
Carrier: SQ Singapore Airlines Limited  
Connection: 0001  
Departure: SFO San Francisco International Airport  
Destination: SIN Singapore Changi Airport
```

Your result: Your uploaded screenshot shows the exercise working: the console displays the full airport names for FRA, JFK, MIA, HAV, SFO, and SIN.

18. Key Concepts to Remember

Concept	Meaning	Example
Structure type	Defines one record	st_airport
Table type	Defines many records	tt_airports
Internal table	Stores multiple rows	airports
CLASS-DATA	One shared class-level attribute	CLASS-DATA airports

Class constructor	Initializes class-level data	SELECT ... INTO TABLE @airports
Table expression	Finds a row in an internal table	airports[airportid = ...]
Inline declaration	ABAP derives the data type	DATA(departure) = ...

19. Exercise 15 Checklist

- ✓ Copied or continued from Exercise 14.
- ✓ Created st_airport.
- ✓ Created tt_airports.
- ✓ Declared CLASS-DATA airports.
- ✓ Generated class_constructor.
- ✓ Loaded /DMO/I_Airport into airports.
- ✓ Used table expressions to find departure and destination airports.
- ✓ Used departure-Name and destination-Name in the output.
- ✓ Activated without errors.
- ✓ Executed successfully and verified the console output.

20. What You Should Understand Before Moving On

Do not move on merely because the code runs. You should be able to explain this sentence in your own words: 'The class constructor loads all airport records into the static internal table airports; get_output then uses the airport ID stored in details to find the matching row and read its Name component.' If you cannot explain that, the syntax is memorized rather than understood.

21. Evidence From Your Practice

The screenshot below is the final execution state from your practice. It shows the class definition and the successful ABAP Console output with airport names.

The screenshot displays the Eclipse IDE with the ABAP class definition for `ZCL_9309_LOCAL_CLASS` in the editor. The class is a local class for the `LCL_CONNECTION` interface. It includes a public section with a `CLASS-DATA` declaration for `com_counter` (type `i`, READ-ONLY) and a `CLASS-METHODS` declaration for `class_constructor`. The `class_constructor` method is implemented with a `IMPORTING` clause for `i_carrier_id` (type `/dmo/carrier_id`) and `i_connection_id` (type `/dmo/connection_id`), and a `RAISING` clause for `cx_abap_invalid_value`. The `get_output` method is implemented with a `RETURNING` clause for `VALUE(r_output)` (type `string_table`). The `PROTECTED SECTION` contains a `PRIVATE SECTION` with a `TYPES` declaration for `st_details` and a `BEGIN OF st_details` block. The `st_details` block contains fields for `Departureairport` (type `/dmo/airport_from_id`), `Destinationairport` (type `/dmo/airport_to_id`), `AirlineName` (type `/dmo/carrier_name`), and `END OF st_details`. The `TYPES` declaration for `tt_airports` is also present. The `BEGIN OF st_details` block is followed by a `DATA` declaration for `st_details` (type `st_details`). The `END OF st_details` block is followed by a `DATA` declaration for `tt_airports` (type `tt_airports`). The `END OF CLASS` statement is at the bottom.

The console output shows the successful execution of the class constructor, displaying the following information:

```

ABAP Console
-----
Carrier: LH Deutsche Lufthansa AG
Connection: 0400
Departure: FRA Frankfurt Airport
Destination: JFK John F. Kennedy International Airport
-----
Carrier: AA
Connection: 0017
Departure: MIA Miami International Airport
Destination: MNY José Martí International Airport
-----
Carrier: SQ Singapore Airlines Limited
Connection: 0001
Departure: SFO San Francisco International Airport
Destination: SIN Singapore Changi Airport

```

Your completed Exercise 15 execution result.